AI ARCHITECTURE SERIES

50+ AI ARCHITECTURE TRADE-OFFS

- Master **production AI system decisions** .
- Ace **Principal & Lead AI Architect interviews** .

RAG VS FINE-TUNE	VLLM VS TRITON	FP8 VS INT4
KNOWLEDGE	SERVING INFRA	QUANTIZATION

Swipe to explore trade-off matrix →

RAG vs Fine-Tuning: How do you choose between external retrieval augmentation and weight parameter updates?

RAG injects dynamic external knowledge at inference time, while Fine-Tuning bakes domain style, syntax, and task behavior directly into model weights.

Retrieval-Augmented Generation (RAG):

- **Pros:** Zero retraining cost, real-time knowledge updates, sub-second source attribution.
- **Cons:** High context window token costs, retrieval latency overhead (50–200ms).

Domain Fine-Tuning (LoRA / SFT):

- **Pros:** Tailored output format/tone, lower latency, smaller prompt token footprint.
- **Cons:** Prone to hallucination on new dynamic facts, high GPU retraining cost.

Decision Rule:

Use **RAG** for fast-changing enterprise data; use **Fine-Tuning** to teach specialized formats, JSON schemas, or niche domain language.

vLLM vs TensorRT-LLM: Which inference engine should you deploy for high-throughput LLM serving?

Selecting the right LLM inference engine dictates GPU server hardware requirements and overall cloud infrastructure bill.

vLLM (PagedAttention Engine):

- **Pros:** Extremely easy Python/C++ setup, native OpenAI API compatibility, near-zero KV cache fragmentation.
- **Cons:** Slightly lower peak single-GPU throughput compared to heavily compiled CUDA kernels.

NVIDIA TensorRT-LLM:

- **Pros:** Maximum hardware efficiency, FP8 kernel acceleration, top p99 latency performance.
- **Cons:** Complex C++ compilation pipelines, slow model iteration cycle.

Decision Rule:

Use **vLLM** for fast dev cycles & standard deployments; switch to **TensorRT-LLM** for ultra-high QPS enterprise workloads on NVIDIA H100s.

Redis vs Aerospike: Designing the online feature store layer for sub-10ms real-time ML scoring.

Feature stores require sub-millisecond point lookups under heavy concurrency to feed inference models without bottlenecking SLAs.

Redis Cluster (RAM-First Store):

- **Pros:** Ultra-low p99 latency (<2ms), rich data structure support, universal developer ecosystem.
- **Cons:** Expensive RAM scaling costs when storing multi-terabyte feature sets.

Aerospike (Hybrid Memory / Flash):

- **Pros:** Index in RAM with data on direct NVMe SSDs, reducing hardware cost by 60-80%.
- **Cons:** Higher operational complexity, strict schema requirements.

Decision Rule:

Use **Redis** for dataset sizes under 500GB; transition to **Aerospike** when feature storage exceeds terabytes.

Pinecone vs Milvus: Managed SaaS Vector DB vs Self-Hosted Open Source on Kubernetes.

Vector databases index high-dimensional embeddings for ANN (Approximate Nearest Neighbor) similarity search in RAG applications.

Pinecone (Serverless SaaS):

- **Pros:** Zero infrastructure overhead, instant serverless scaling, integrated metadata filtering.
- **Cons:** Vendor lock-in, recurring SaaS cost scales rapidly with high write throughput.

Milvus / Qdrant (Self-Hosted OSS):

- **Pros:** Total data privacy/VPC deployment, customizable indexing algorithms (HNSW vs IVF_FLAT).
- **Cons:** Requires dedicated Kubernetes ops team to manage sharding and memory nodes.

Decision Rule:

Start with **Pinecone** for fast MVP delivery; migrate to **Milvus** for strict HIPAA/SOC2 compliance or massive vector scale.

FP8 vs INT4 AWQ: Balancing precision loss against VRAM memory footprints for LLM deployment.

Quantization compresses model weights, enabling large LLMs to run on fewer GPUs with reduced memory bandwidth limits.

FP8 (8-bit Floating Point):

- **Pros:** Native hardware support on H100 GPUs, near-zero perplexity loss compared to FP16.
- **Cons:** 2× VRAM footprint compared to INT4; requires modern Ada/Hopper architecture.

INT4 AWQ (Activation-aware Weight Quantization):

- **Pros:** Fits 70B models onto single 48GB GPU; cuts memory bandwidth bottlenecks by 4×.
- **Cons:** Minor accuracy degradation on complex mathematical & coding tasks.

Decision Rule:

Use **FP8** for mission-critical reasoning tasks; use **INT4 AWQ** for high-throughput cost-sensitive chat endpoints.

REST / gRPC vs Async Kafka Streams: Choosing client-facing inference transport patterns.

Architecting payload transport layers determines system resilience under high concurrency and traffic spikes.

Synchronous gRPC / REST APIs:

- **Pros:** Immediate response payload delivery, simpler client implementation, low overhead.
- **Cons:** Prone to cascading timeouts if model inference experiences latency spikes.

Asynchronous Kafka / Event-Driven Queues:

- **Pros:** Natural load buffering, zero dropped queries during peak spikes, easy retry handling.
- **Cons:** Requires polling or WebSocket infrastructure for client response delivery.

Decision Rule:

Use **gRPC** for real-time user-facing chat apps; use **Kafka** for heavy batch processing or background document parsing.

Monolithic 70B Model vs Semantic Router + Speculative Micro-Models: Cost vs Complexity.

Routing requests to small specialized models reduces compute costs compared to passing every query to a large LLM.

Monolithic 70B / 405B Model Fleet:

- **Pros:** Simplified architecture, single API deployment, high generalized reasoning across all query types.
- **Cons:** Massive compute cost; simple classification queries burn expensive GPU cycles.

Semantic Router + Micro-Model Cascade:

- **Pros:** 80% cost reduction by routing simple queries to 8B models or BERT classifiers.
- **Cons:** Increased pipeline complexity, router maintenance, potential routing errors.

Decision Rule:

Deploy a **Semantic Router** when request volume exceeds 1M queries/day and query complexity varies widely.

On-Prem Reserved GPUs vs Cloud Serverless GPU Instances: CapEx vs OpEx economics.

GPU infrastructure strategy dictates profit margins for AI SaaS applications operating at high scale.

Reserved GPU Clusters (On-Prem / Bare-Metal):

- **Pros:** Lowest unit cost per GPU hour, zero cloud markup, complete data sovereignty.
- **Cons:** High upfront capital expenditure (CapEx), fixed capacity limits during sudden demand spikes.

Serverless / On-Demand Cloud GPUs:

- **Pros:** Pay-per-second execution, infinite elasticity, zero idle server overhead.
- **Cons:** 3×-5× price markup per GPU hour; susceptible to cloud stockouts.

Decision Rule:

Use **Serverless GPUs** for unpredictable startup traffic; switch to **Reserved Bare-Metal** once GPU utilization stabilizes above 70%.

Batch Offline Processing vs Real-Time Streaming Feature Computation: Freshness vs Cost.

Choosing when features are computed directly impacts model accuracy and stream processing complexity.

Batch Feature Pipelines (Spark / Airflow):

- **Pros:** High throughput, cheap compute, easy debugging & deterministic backfills.
- **Cons:** Feature staleness (hours/days), unable to capture immediate user intent shifts.

Real-Time Streaming Pipelines (Flink / Kafka):

- **Pros:** Sub-second feature freshness, immediate adaptation to fraud or user clicks.
- **Cons:** Complex stateful stream management, expensive 24/7 cluster operations.

Decision Rule:

Use **Batch** for demographic & historical features; use **Streaming** for fraud detection & real-time search recommendations.

Dense Embeddings vs Hybrid Sparse-Dense Search (SPLADE + BM25): Retrieval precision trade-offs.

Vector search alone often struggles with exact keyword matches, part numbers, and specialized acronyms.

Dense Vector Retrieval (Cosine / Dot Product):

- **Pros:** Captures semantic intent, synonyms, and multi-lingual concept similarity.
- **Cons:** Fails on exact product SKUs, code symbols, and specialized terminology.

Hybrid Sparse-Dense Search (BM25 + SPLADE):

- **Pros:** Combines semantic understanding with exact keyword match accuracy.
- **Cons:** Requires Reciprocal Rank Fusion (RRF) reranking step & double index storage.

Decision Rule:

Use **Hybrid Search** for enterprise docs, e-commerce catalog search, and legal document retrieval.

Rule-Based Guardrails vs LLM-as-a-Judge Security Models: Safety vs Speed & Cost.

Protecting LLM applications against prompt injections and toxic outputs requires layered guardrail filtering.

Rule-Based & Regex Filters (NeMo / Llama-Guard Rules):

- **Pros:** Sub-millisecond execution, deterministic blocking, zero API cost.
- **Cons:** Easily bypassed by sophisticated prompt adversarial obfuscations.

LLM-as-a-Judge Guardrail Models:

- **Pros:** Deep semantic contextual analysis, detects nuanced policy violations.
- **Cons:** Adds 100-300ms latency overhead and doubles per-query inference costs.

Decision Rule:

Deploy **Rule Filters** on input ingress; run **LLM-as-a-Judge** asynchronously on high-risk output payloads.

LoRA / QLoRA Adapters vs Full Parameter Fine-Tuning: Parameter efficiency trade-offs.

Updating full model weights requires massive GPU clusters, whereas Parameter-Efficient Fine-Tuning (PEFT) trains lightweight adapters.

LoRA / QLoRA (Rank-Decomposed Adapters):

- **Pros:** Trains <1% of parameters, fits on single consumer GPU, easy multi-tenant adapter swapping.
- **Cons:** Slightly lower capacity for drastic domain knowledge changes.

Full Parameter Fine-Tuning:

- **Pros:** Maximum model capability adaptation, updates all layer representations.
- **Cons:** Requires 8× VRAM, expensive distributed training infra, risk of catastrophic forgetting.

Decision Rule:

Use **LoRA/QLoRA** for 95% of enterprise adaptation tasks; reserve **Full Fine-Tuning** for building base domain models.

Centralized Feature Store vs Decentralized Microservice Data Layer: Governance vs Team Autonomy.

Decoupling data pipelines across engineering teams requires balancing central feature reuse against microservice independence.

Centralized Feature Store (Feast / Tecton):

- **Pros:** Eliminates feature re-computation across teams, guarantees training-serving feature alignment.
- **Cons:** Centralized governance bottleneck, complex cross-team schema coordination.

Decentralized Microservice Data Layer:

- **Pros:** Full team autonomy, independent deployment cycles, tailored data schemas.
- **Cons:** High feature duplication, risk of training-serving skew across teams.

Decision Rule:

Adopt a **Central Feature Store** when 3+ ML teams share overlapping user/item entity features.

Edge On-Device Inference vs Cloud API Execution: Latency & Privacy vs Compute Power.

Deploying models onto mobile/edge hardware trades off raw model intelligence for zero latency and offline capability.

Edge On-Device Models (CoreML / ExecuTorch):

- **Pros:** Zero network latency, 100% offline privacy, zero server API cost per query.
- **Cons:** Limited to small models (<3B parameters), constrained mobile battery/RAM limits.

Cloud API Execution (Centralized GPU Clusters):

- **Pros:** Access to massive frontier models (70B+), continuous model updates without app releases.
- **Cons:** Network latency dependency, recurring cloud compute infrastructure bill.

Decision Rule:

Use **Edge Models** for real-time camera/voice UI tasks; route complex reasoning queries to **Cloud APIs**.

Continuous Batching vs Static Batching: Serving engine throughput optimization choices.

LLM text generation outputs dynamic sequence lengths, making traditional static request batching highly inefficient.

Static Batching (Fixed Dimension Batches):

- **Pros:** Simple execution model, predictable tensor shapes for fixed CUDA kernels.
- **Cons:** Severe GPU idling while waiting for the longest sequence in batch to finish generation.

Continuous Iteration-Level Batching (vLLM / Orca):

- **Pros:** Instantly inserts incoming requests as soon as completed queries finish, boosting throughput 230%.
- **Cons:** Complex dynamic memory allocation & KV cache management.

Decision Rule:

Always use **Continuous Batching** for dynamic LLM generation workloads to maximize GPU utilization.

50+ TRADE-OFFS LIKE THIS

Master Production AI Infrastructure & Architecture Decisions.

WHAT WE COVERED:

- → 15 Real-World AI Architecture Trade-Offs
- → RAG vs Fine-Tuning & Vector DB Trade-offs
- → Serving Engine & GPU Infra Unit Economics

Save & Repost for your next interview!
Follow Resnur AI for daily system design breakdowns.